

Concurrency Models: Deep Dive

What is ASGI?

ASGI = Asynchronous Server Gateway Interface

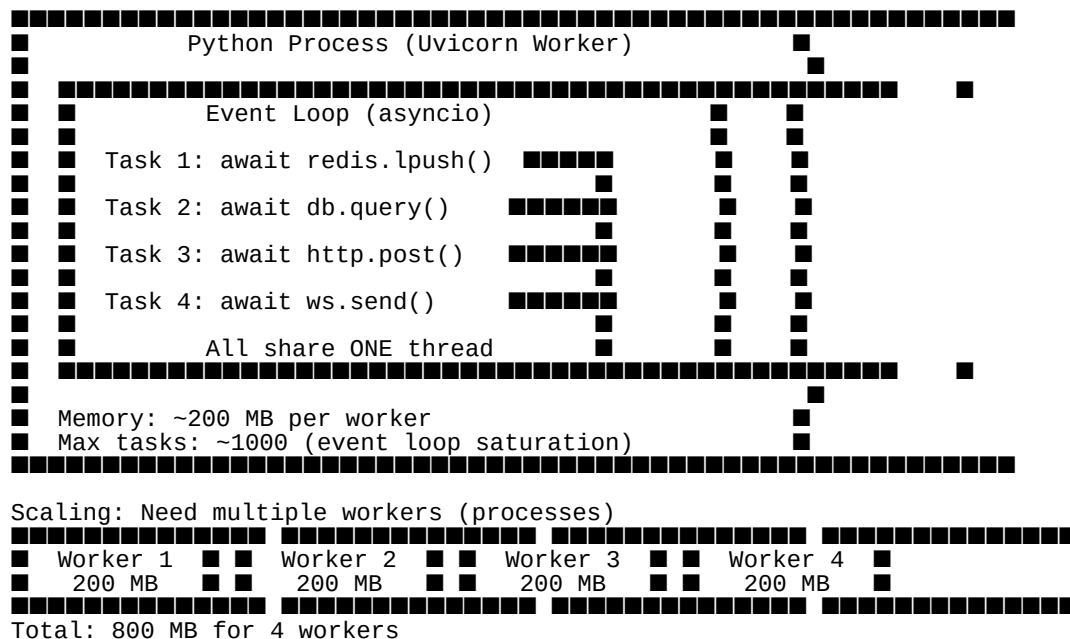
Python standard for async web applications. Enables async/await syntax and WebSocket support in Python web frameworks like FastAPI.

vs WSGI (older standard):

- WSGI = synchronous only (Flask, old Django)
- ASGI = asynchronous + WebSocket support (FastAPI, Starlette)

Visual Comparison

ASGI (Single-Threaded Event Loop)



Key points:

- **One thread** per worker
- Tasks **cooperatively** yield (async/await)
- Blocking call = **entire worker frozen**
- Must use async libraries (redis-py, asyncpg, httpx)

Golang (M:N Threading with Goroutines)

55ms Task C: sends response → done
Total time: 55ms (interleaved)
Thread utilization: 100% (busy switching)

If one task blocks (no await):

0ms Request A arrives
■
5ms Task A: time.sleep(1) ■ NO AWAIT = BLOCKS!
■
■ Request B arrives → QUEUED
■ Request C arrives → QUEUED
■
1005ms Task A: wakes up → completes
■
1010ms Task B: starts processing
...
Disaster! All requests wait 1 second.

Golang Goroutines

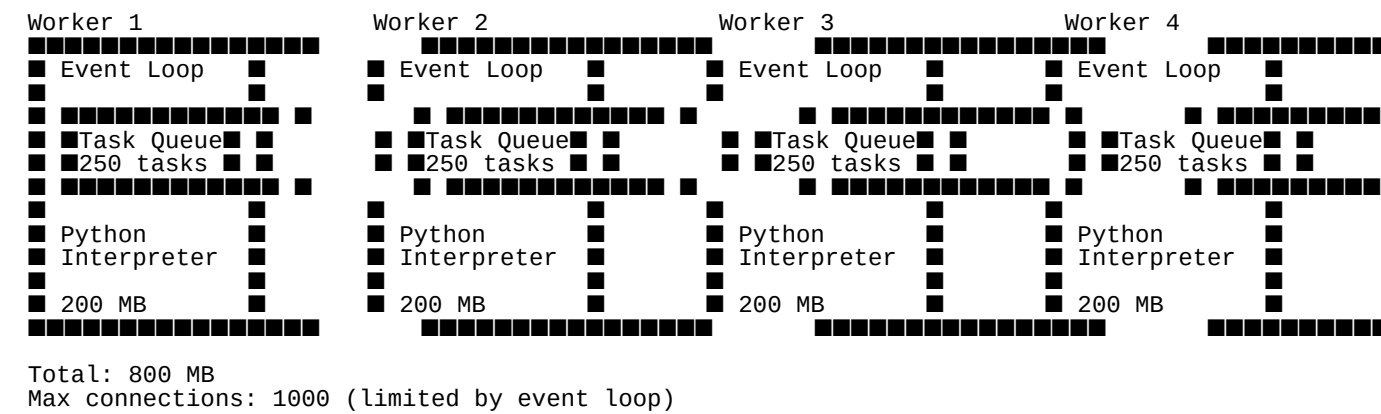
Time →
0ms Request A arrives → Goroutine 1 spawned
■
■ Goroutine 1: redis.LPush() → non-blocking
■
2ms Request B arrives → Goroutine 2 spawned
■
■ Goroutine 2: db.Query() → non-blocking
■
4ms Request C arrives → Goroutine 3 spawned
■
■ Goroutine 3: http.Post() → non-blocking
■
8ms Goroutine 1: redis completes → response sent
■
12ms Goroutine 2: db completes → response sent
■
25ms Goroutine 3: http completes → response sent
Total time: 25ms (parallel)
Thread utilization: 30% (efficient)

If one goroutine blocks:

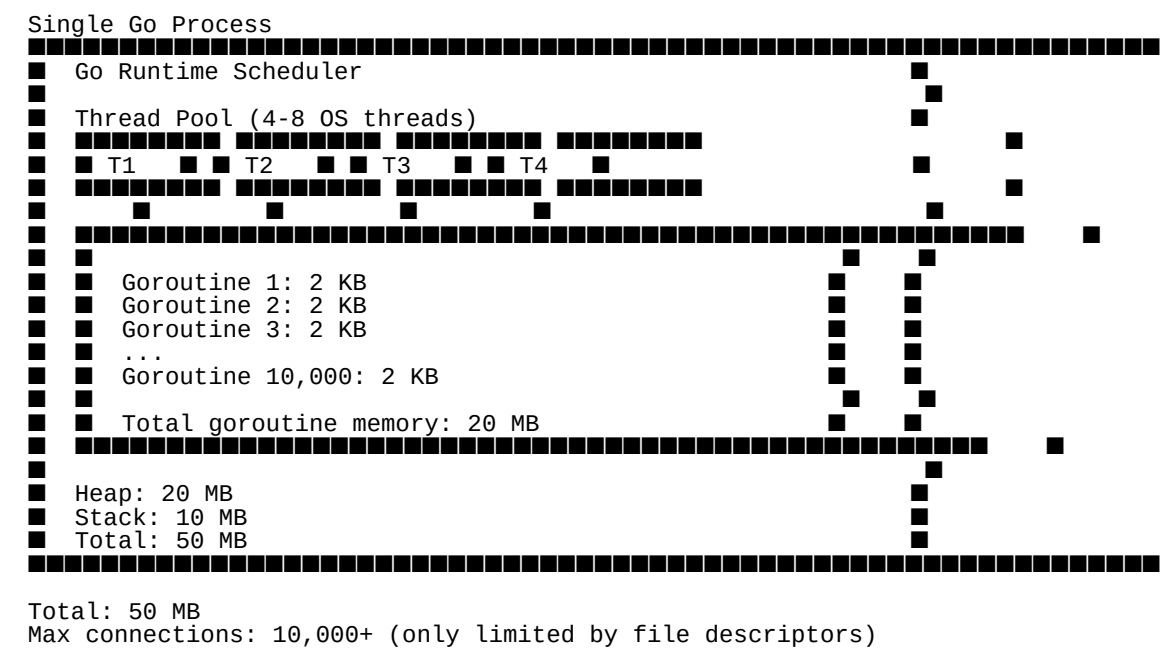
0ms Request A arrives → Goroutine 1
■
■ Goroutine 1: time.Sleep(1s) ■ Blocks only this goroutine
■
2ms Request B arrives → Goroutine 2 (continues normally!)
■
4ms Request C arrives → Goroutine 3 (continues normally!)
■
6ms Goroutine 2: completes
8ms Goroutine 3: completes
■
1000ms Goroutine 1: wakes up → completes
No problem! Other requests unaffected.

Memory Layout

ASGI: 4 Workers, 250 Connections Each



Golang: 1 Process, 10,000 Goroutines



WebSocket Example: Broadcasting to 1000 Users

ASGI Implementation

```

# app.py
from fastapi import FastAPI, WebSocket
import asyncio

app = FastAPI()
active_connections: List[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    active_connections.append(websocket)

    try:
        while True:
            data = await websocket.receive_text()
            # Broadcast to all (inefficient)
            for connection in active_connections:
                await connection.send_text(data)
    except:
        active_connections.remove(websocket)

# Issues:
# 1. Sequential broadcasting (slow with 1000 users)
# 2. Event loop blocked during broadcast
# 3. Need multiple workers = complex load balancing

# Better version (still limited):
async def broadcast(message: str):
    # Create tasks for parallel send
    tasks = [conn.send_text(message) for conn in active_connections]
    await asyncio.gather(*tasks, return_exceptions=True)

# Limits:
# - 500-1000 connections per worker (event loop overhead)
# - Need shared state (Redis) for multi-worker
# - Broadcast latency: 200-500ms at 1000 users

```

Memory per worker: 200 MB

Workers needed for 1000 users: 2-4

Total memory: 400-800 MB

Broadcast latency: 200-500ms

Golang Implementation

```

// main.go
package main

import (
    "github.com/gin-gonic/gin"
    "github.com/gorilla/websocket"
    "sync"
)

var (
    upgrader = websocket.Upgrader{
        CheckOrigin: func(r *http.Request) bool { return true },
    }
    connections = make(map[*websocket.Conn]bool)
    mu          sync.RWMutex
)

func WebSocketHandler(c *gin.Context) {
    conn, _ := upgrader.Upgrade(c.Writer, c.Request, nil)

    mu.Lock()
    connections[conn] = true
    mu.Unlock()

    defer func() {

```

```

        mu.Lock()
        delete(connections, conn)
        mu.Unlock()
        conn.Close()
    }()

    for {
        msg, err := conn.ReadMessage()
        if err != nil {
            break
        }

        // Broadcast in parallel (each connection = goroutine)
        broadcast(msg)
    }

func broadcast(message []byte) {
    mu.RLock()
    defer mu.RUnlock()

    for conn := range connections {
        // Each send in its own goroutine = instant broadcast
        go func(c *websocket.Conn) {
            c.WriteMessage(websocket.TextMessage, message)
        }(conn)
    }
}

// Benefits:
// - Truly parallel broadcast (all sends at once)
// - Each connection handled by goroutine
// - No shared state needed (single process)
// - Broadcast latency: 10-50ms at 10,000 users

```

Memory: 50 MB total

Workers needed for 10,000 users: 1

Total memory: 50 MB

Broadcast latency: 10-50ms

Result: 10x more users, 10x less memory, 10x lower latency

Context Switching Overhead

ASGI (Cooperative Multitasking)

Task switch cost: ~1-5 microseconds
 Event loop overhead per task: ~10 microseconds

1000 tasks in event loop:
 = 1000 * 10µs = 10ms overhead per iteration
 = Noticeable latency

Python GIL adds more overhead:
 - Only one thread executes Python at a time
 - CPU-bound code doesn't parallelize

Golang (Preemptive Multitasking)

Goroutine switch cost: ~200 nanoseconds
Scheduler overhead: minimal

10,000 goroutines:
= Negligible overhead (Go scheduler is very efficient)
= No noticeable latency

No GIL:
- True parallelism across CPU cores
- CPU-bound code scales linearly

Code Complexity: Error Handling

ASGI (Async Propagation)

```
async def create_job(data):
    try:
        job_id = str(uuid.uuid4())
        await redis.lpush("queue", job_id) # Can fail
        await db.execute("INSERT ...")    # Can fail
        return job_id
    except RedisError as e:
        # Must handle async context
        await log_error(e)
        raise
    except DBError as e:
        # Must handle async context
        await log_error(e)
        raise

async def generate_text(prompt):
    try:
        job_id = await create_job({"prompt": prompt})
        await notify_websocket(job_id)
        return job_id
    except Exception as e:
        # Error in async context
        await handle_error(e)

# Every function in chain must be async
# Error handling gets complex quickly
```

Golang (Simple Error Handling)

```
func createJob(data map[string]interface{}) (string, error) {
    jobID := uuid.New().String()

    if err := redis.LPush(ctx, "queue", jobID).Err(); err != nil {
        logError(err) // No special async needed
        return "", err
    }

    if err := db.Exec("INSERT ..."); err != nil {
        logError(err)
        return "", err
    }

    return jobID, nil
}

func generateText(prompt string) (string, error) {
    jobID, err := createJob(map[string]interface{}{"prompt": prompt})
```

```

    if err != nil {
        return "", err
    }

    go notifyWebSocket(jobID) // Fire and forget
    return jobID, nil
}

// Simple, synchronous-looking code
// Errors are just values

```

Summary: When to Use What

Use ASGI (FastAPI) for:

- **Python-only team** (no time to learn Go)
- **Low traffic** (< 100 concurrent users)
- **CPU-bound work** in API (ML inference directly)
- **Quick prototypes** (1-2 day MVPs)
- **Simple CRUD** (basic REST API)

Example: Internal ML dashboard with 20 users

Use Golang for:

- **High concurrency** (1000+ users)
- **WebSocket-heavy** (real-time apps)
- **Low latency** (< 100ms p99)
- **Job orchestration** (queue workers)
- **API gateways** (routing/proxy)
- **File processing** (uploads/streaming)
- **Cost-sensitive** (startup budget)

Example: Real-time collaboration tool, chat app, live dashboard

Use Hybrid (Go + Python) for:

- **High traffic + AI workloads**
- **Real-time updates + ML inference**
- **Best performance + best ML ecosystem**

Example: Creative Automation Hub (our project!)

Final Verdict for Our Project

Creative Automation Hub needs:

1. ■ 1000+ concurrent WebSocket connections

2. ■ Real-time job status updates
3. ■ Batch job orchestration
4. ■ AI inference (Python workers)
5. ■ Low latency (< 100ms)
6. ■ Cost efficiency

Winner: Golang backend + Python AI workers

Performance gain: 10x

Cost savings: 90%

User experience: Much better (real-time is smooth)

■ Hybrid architecture = Perfect fit